NetSocratic: Intent Disambiguation for Network Configuration

Abstract

Network misconfiguration remains a primary cause of large-scale outages and security breaches. While existing research has produced sophisticated tools to verify or synthesize configurations from formal, declarative specifications, these tools often assume the input specification is already a perfect reflection of the operator's goal. In practice, high-level intents like "ensure redundancy" suffer from a *Specification Gap*—where natural language is too ambiguous for direct translation—and an *Oracle Problem*, where operators provide incomplete requirements. We introduce *NetSocratic*, a multi-agent framework that shifts the focus to the intent itself by treating disambiguation as an interactive search problem. Our system employs a *Clarification Agent* to resolve linguistic vagueness and a *Generator Agent* to synthesize candidate configurations. A *Selection Agent* then leverages relational network verification to identify behavioral differences between these candidates that LLM might have missed, transforming them into targeted, minimal questions that allow the operator to prune inconsistent interpretations. This report details our multi-agent architecture, provides an evaluation plan for a dataset of 1,425 intents, and outlines a roadmap for a fully integrated, self-recovering intent disambiguation pipeline based on our initial proof-of-concept findings.

1 Problem

Networks are complex distributed systems that remain notoriously difficult to implement and maintain. While abstractions like Software-Defined Networking (SDN) have significantly simplified resource management and routing, they have not yet insulated the network from human-induced failures. Misconfigurations and logic errors continue to trigger massive outages and security breaches, costing organizations millions of dollars annually [7, 11, 12, 17].

In response, the community has developed sophisticated *network verification* [3, 9, 13–15] and *synthesis* [4, 5, 8, 10, 16] tools. Verification allows operators to define declarative invariants that are automatically checked against underlying configurations, while synthesis tools translate high-level specifications directly into low-level device settings. However, these tools suffer from two critical limitations that prevent them from addressing issues in high-level intents: (i) they require intents to be expressed in formal, logic-based languages that are often difficult to author and debug [6], and (ii) they lack support for *substrate-independent* intents. A substrate-independent intent is a goal defined purely by business logic (the "what") rather than the physical network layout (the "how"). Most high-level objectives are inherently abstract—for example, *"isolate Payroll servers from the Guest Wi-Fi"*—yet current tools require these to be manually translated into topology-specific details such as IP prefixes, VLAN tags, or BGP neighbor exports. This forces the operator to manage the inherent complexity of the intent and the underlying physical substrate simultaneously.

Consequently, a significant manual gap remains. Because most network goals originate as informal natural language, they must undergo an *implicit translation* into technical specifications—a process that relies heavily on an operator's "tribal knowledge" of local constraints and undocumented history. Currently, there is no structured way to formalize these interpretative decisions. As a result, any error introduced during the initial translation persists, silently undermining the guarantees of robust verification tools.

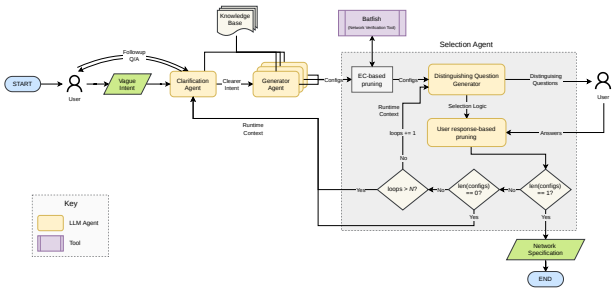

Figure 1: NetSocratic's System Architecture.

As the industry moves toward network automation, this lack of a structured method to handle intent-level errors becomes a critical bottleneck. Automation increases the velocity of deployment, meaning a misinterpreted intent can be propagated across a global infrastructure instantly, possibly without the traditional "sanity check" of a manual rollout. While we can mathematically guarantee that a configuration matches a specification, we still have no mechanism to ensure that the specification matches the user's goal.

Hence, in order to bridge the gap between high-level intent and formal specification, we focus on two key challenges:

1. The Specification Gap (Ambiguity): High-level intents are inherently underspecified and subject to multiple valid interpretations. For example, a directive to *"isolate guest traffic"* could be correctly realized via VLAN tagging, firewall ACLs, or Virtual Routing and Forwarding (VRFs), where the "best" choice depends entirely on unstated local security postures. Currently, these goals undergo a one-time, informal translation into a technical specification. This process is problematic because it forces a definitive choice among competing interpretations without a formal mechanism to document the reasoning. Consequently, the original ambiguity is not resolved but merely suppressed; the discarded interpretations are lost, leaving no structured way to audit or correct the specification if the operator's initial interpretative leap proves semantically incorrect for the given context.

2. The Oracle Problem (Requirement Incompleteness): Operators often act as "imperfect oracles," providing requirements that are functionally incomplete or logically inconsistent. For instance, an operator may specify a policy to *"prevent all cross-departmental traffic,"* failing to account for shared infrastructure services like a common backbone. While the resulting configuration is technically correct according to the provided rule, it inadvertently collapses required network connectivity. Such omissions are latent; because the operator takes these "common sense" invariants for granted, the error only becomes apparent when they are confronted with concrete evidence of failure, such as an unexpected reachability drop in a data plane. Consequently, existing tools face a hard ceiling on correctness: they can guarantee the configuration matches

the input, but they cannot verify the input against the operator's unarticulated expectations.

2 Solution

NetSocratic addresses ambiguous network intent as an iterative search and elimination problem. Rather than committing immediately to a single formalization, the system maintains a set of plausible candidate configurations and progressively narrows this set until it identifies the configuration that best matches the operator's intent. The architecture proceeds in four stages: (i) clarifying the user's intent in natural language, (ii) generating candidate network configurations consistent with that clarified intent, (iii) pruning behaviorally redundant candidates by comparing them exactly across attribute-wise equivalence classes (ECs) using a relational network verification tool [2], and (iv) asking the user as few distinguishing questions as possible to determine which remaining configuration matches their preference. This decomposition lets the system use language models where semantic ambiguity is obvious while relying on exact network analysis for behavioral comparison and pruning of subtle differences in interpretations. NetSocratic's system architecture is shown in Fig 1. We detail it out below.

Clarification Agent As shown in Figure 1, the system begins when the user provides a vague intent in natural language, such as:

> *"Ensure connectivity between athens and our servers. Route through london if possible. Make sure there's redundancy."*

This input is first handled by the *Clarification Agent*, whose role is to resolve ambiguity that is still best expressed in human language and to gather enough missing detail that the downstream Generator Agent can synthesize a single configuration in one run. For this example, the agent may ask follow-up questions such as:

> *"(i) Which exact prefixes are meant by 'our servers'? (ii) What exact redundancy level is required for paths where we have to 'make sure there's redundancy'?"*

Using the user's answers together with a knowledge base (e.g., the topology, base configurations, documentation), the Clarification Agent produces a constrained, clarified intent representation to pass downstream. In this running example, the clarified intent becomes:

> *"Athens must be able to reach 100.0.29.0/24. Traffic from Athens should prefer paths that go through London. The routing must provide exactly 3 load-balanced paths, ensuring redundancy."*

Generator Agent. The clarified intent is then passed to the *Generator Agent*. The Generator Agent is designed to produce one configuration that it believes best satisfies the clarified intent. It is also able to access the Knowledge Base used by the Clarifying Agent. In our current setup, the Generator Agent produces *OSPF* configurations. To construct a candidate set for downstream selection, NetSocratic runs the Generator Agent multiple times, including with different temperatures and sampling conditions, and collects the resulting configurations. These are then passed to the Selection Agent for exact behavioral comparison and pruning.

Selection Agent. Inside the Selection Agent, the first step is *EC-based pruning*. Here, a verification tool (Batfish [2]) is used to compute exact behavioral comparisons across distinct *attribute-wise equivalence classes (ECs)*. This allows the system to determine when two candidate configurations are indistinguishable with respect to a specific policy dimension, such as reachability, waypointing, or load balancing. Instead of comparing configurations as monolithic blocks, the agent evaluates them per-attribute; if multiple candidates induce identical behavior for a particular attribute, they are considered redundant within that dimension, and the set is pruned to keep only unique behavioral representatives. In the running example, two different low-level configurations might both ensure reachability from Athens to 100.0.29.0/24 but differ in their load-balancing logic; the Selection Agent identifies this overlap per-attribute and only retains configurations that offer distinct behavioral profiles. This stage is deterministic (no LLMs), ensuring pruning is based strictly on formal Batfish-based comparison.

After EC-based pruning, the surviving candidates are passed to the *Distinguishing Question Generator*, which searches for concrete behavioral differences among them and converts those differences into user-facing questions. The associated selection logic chooses questions that are expected to eliminate as many remaining candidates as possible, thereby minimizing the total number of user interactions. For example, if two surviving configurations differ only in whether the London requirement is mandatory or merely preferred under congestion, the system asks a targeted question about whether traffic from Athens to 100.0.29.0/24 must always traverse London or is bypassing London acceptable to preserve connectivity. The user's answer is then consumed by the *user response-based pruning* module, which removes every candidate inconsistent with that answer. This mechanism also addresses the *oracle problem*: by confronting the operator with specific, divergent behaviors they may not have originally considered, the system forces the articulation of latent requirements. If an operator realizes that none of the presented behaviors match their actual goal, it serves as a trigger to refine the initial intent. If exactly one configuration remains, the system outputs it as the final *Network Specification*; otherwise, the question-generation and pruning loop repeats.

The architecture also includes explicit recovery paths for cases where the current candidate set cannot be resolved cleanly. If the user, during user-response based pruning, informs NetSocratic that no candidate specification matches their requirement, or if the number of interaction rounds exceeds a preset limit N, the system packages the current runtime context and sends it back to the Clarification Agent. This runtime context records what has already been generated, compared, and ruled out, allowing the next clarification pass to ask better-targeted questions and regenerate a more appropriate candidate configuration set from the Generator Agent. Consequently, NetSocratic is a closed-loop system in which natural-language clarification, candidate generation, EC-based network analysis, and minimal-question interaction work together to converge on the operator's preferred configuration. Out system implementation is available at [1]

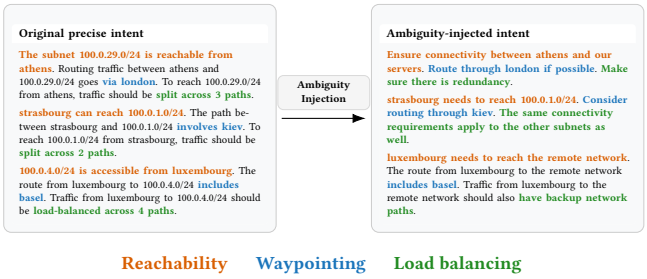

Figure 2: Transforming a precise high-level network intent into an ambiguity-injected version.

3 Evaluation Setup and Preliminary Results

Dataset. We build our evaluation dataset on top of NetConfEval[18], a benchmark for studying whether LLMs can translate high-level network requirements in natural language to machine-readable specifications. In the NetConfEval translation task, the target specification captures three key policy dimensions—reachability, waypointing, and load balancing. We select only those intents which have implications along all 3 policy dimensions. After this initial filtering we are left with a dataset of 1425 intents. We reuse the first two fields of this data: the original high-level intent and its correct formal specification. The first field serves as the seed operator intent, while the second is treated as the ground truth semantic target. Since the original NetConfEval-style intents are quite well-defined, they do not capture the ambiguity found in real operator inputs. To address this, we introduce an LLM-based ambiguity injection step. We prompt an LLM to rewrite each clear intent so that it becomes more vague while preserving the same underlying semantics. The prompt explicitly asks the model to introduce ambiguity along three dimensions: reachability, waypointing, and load balancing. Concretely, this includes replacing exact endpoint references with vague phrases such as "the remote network," softening hard waypoint constraints into preferences such as "route through X if possible," and converting path-count requirements into underspecified statements such as "ensure redundancy". The resulting output is stored as the ambiguous intent, while the original formal specification is retained as the ground-truth interpretation. Each generated ambiguous intent was manually audited to confirm its semantic plausibility in practice. Figure 2 shows exact examples of precise intents, with their corresponding ambiguous intents we've used for our evaluation.

Experimental Methodology. For the first phase of the project, we use a pilot subset of 5 ambiguous examples from the dataset to validate the end-to-end workflow. This smaller sample lets us manually inspect and verify whether the system correctly identifies the vague information in the intent and asks useful questions to the users in both the clarification and the selection phases. We used `gpt-5.3-codex` for the Clarification and the Generator Agents while we used Gemini (Fast Mode) in the Selection Agent. We emulate user interactions with NetSocratic through an LLM (Gemini, Fast) that has context about the correct intended user configuration from the NetConfEval dataset.

Results. We run each of the 5 vague intents from the ambiguous dataset with NetSocratic and compare the final configuration generated with the correct configuration and report results in Table 1 below. We also provide the count of the number of questions asked to the user by the Clarification Agent and the Selection Agent.

Table 1: Intent Generation and Question Counts

Intent	Correct?	Q (Clarify)	Q (Select)
Intent 1	Yes	5	5
Intent 2	Yes	3	2
Intent 3	?	3	1
Intent 4	Yes	3	0
Intent 5	?	3	2

Observe that NetSocratic produced the correct configuration for 3/5 evaluated intents. For Intents 3 and 5, the system determined that none of the configurations in the generated candidate set matched the user's intended policy. In the full design, this outcome should trigger the recovery loop described in §2, where the Selection Agent passes runtime context back to the Clarification Agent so that the system can gather additional clarification and generate a new candidate set. Since this recovery path is not yet implemented NetSocratic could not continue to a second clarification-and-generation round. Furthermore, we observed that the questions asked to the users by both the Clarifying Agent and the Selection Agent were succinct, and small in number (< 5).

4 Future Work

Our preliminary evaluation serves as a successful proof of concept, demonstrating that NetSocratic effectively bridges the gap between vague or incomplete natural language intent and formal network specifications. Initial results show that the system successfully converges on correct configurations for most intents and accurately identifies when further clarification is needed. To build on this foundation, our future work focuses on the following objectives:

- **End-to-End Pipeline Integration:** Programmatically coupling the *Clarification*, *Generator*, and *Selection* agents into a fully autonomous closed-loop system to remove manual orchestration.
- **Recovery Loop Implementation:** Developing a feedback mechanism that packages the *Runtime Context* (prior failures and Q&A history) to allow the Clarifying Agent to resolve cases where no valid configuration was initially found.
- **Large-Scale Benchmarking:** Scaling the evaluation from the pilot subset to the full dataset of *1,425 intents* to validate system robustness across intents.
- **Baseline Comparison:** Benchmarking NetSocratic against a "Single-Pass" LLM generation approach to quantify the accuracy gains provided by our *Batfish-based* behavioral analysis.
- **Interaction Optimization:** Refining the *Distinguishing Question Generator* to reduce the $InteractionCost$, ensuring the system converges on the correct specification with the fewest possible user queries.

References

[1] [n. d.]. CS598: NetSocratic Codebase. https://github.com/lhan0123/cs598_LMZ_final.

[2] Batfish Project. [n. d.]. Batfish. urlhttps://www.batfish.org/.

[3] Ryan Beckett, Aarti Gupta, Ratul Mahajan, and David Walker. 2017. A General Approach to Network Configuration Verification. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*. 428–441.

[4] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, and David Walker. 2016. Don't Mind the Gap: Bridging Network-wide Objectives and Device-level Configurations. In *Proceedings of the 2016 ACM SIGCOMM Conference*. 328–341.

[5] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, and David Walker. 2017. Network configuration synthesis with abstract topologies. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 437–451.

[6] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, and Todd Millstein. 2023. Lessons from the evolution of the Batfish configuration analysis tool. In *Proceedings of the ACM SIGCOMM 2023 Conference* (New York, NY, USA) *(ACM SIGCOMM '23)*. Association for Computing Machinery, New York, NY, USA, 122–135. doi:10.1145/3603269.3604866

[7] CrowdStrike. 2024. External Post-Incident Review (PIR): Content Configuration Update. (August 2024). Logic error in Channel File 291 leading to out-of-bounds memory access..

[8] Ahmed El-Hassany, Petar Tsankov, Laurent Vanbever, and Martin Vechev. 2018. NetComplete: Practical Network-Wide Configuration Synthesis with Autocompletion. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. 579–594.

[9] Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd Millstein. 2015. A General Approach to Network Configuration Analysis. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. 469–483.

[10] Aaron Gember-Jacobson, Aditya Akella, Ratul Mahajan, and Hongqiang Harry Liu. 2017. Automatically Repairing Network Control Planes Using an Abstract Representation. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*. 359–373.

[11] Google Cloud. 2025. *Google Cloud Infrastructure Components Incident Report*. Technical Report. Incident affecting multiple Cloud services via API crash loops..

[12] Santosh Janardhan. 2021. Understanding the Facebook Outage. *Engineering at Meta* (2021). https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/

[13] Peyman Kazemian, George Varghese, and Nick McKeown. 2012. Header Space Analysis: Static Checking for Networks. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. 113–126.

[14] Ahmed Khurshid, Xuan Zou, Zhou Wenxuan, Matthew Caesar, and P Brighten Godfrey. 2013. VeriFlow: Verifying Network-Wide Invariants in Real Time. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*. 15–27.

[15] Santhosh Prabhu, Kuan-Yen Chou, Ali Kheradmand, Brighten Godfrey, and Matthew Caesar. 2020. Plankton: Scalable Network Configuration Verification through Model Checking. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 953–967.

[16] Sivaramakrishnan Ramanathan, Ying Zhang, Moise Gawish, Yogesh Mundada, Zihui Wang, Sanghee Yun, Eric Lippert, Walid Taha, Minlan Yu, and Jelena Mirkovic. 2023. Aura: Practical Intent-driven Routing Configuration Synthesis. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 507–523.

[17] Rogers Communications. 2022. *Response to CRTC Request for Information: July 8 Outage*. Technical Report. Canadian Radio-television and Telecommunications Commission. Detailing the deletion of a routing filter on distribution routers..

[18] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, and Marco Chiesa. 2024. NetConfEval: Can LLMs Facilitate Network Configuration? *Proc. ACM Netw.* 2, CoNEXT2, Article 7 (June 2024), 25 pages. doi:10.1145/3656296